DEVMIND AI

Codebase Intelligence & Change-Risk Analysis Platform

AI engineering portfolio project
A practical implementation blueprint for a senior full-stack developer targeting AI engineering roles.

STACK Angular | Java 21 | Spring Boot | PostgreSQL + pgvector | Ollama

PLAN Four-week MVP | Approximately 60-100 focused hours | Low-cost-first approach

Purpose

Build a credible AI-enabled engineering product that solves a real developer problem and demonstrates reliable retrieval, program analysis, tool use, evaluation, security, and full-stack architecture.

Guiding principle: build evidence-backed code intelligence first. Add autonomous actions only after the analysis is reliable.

Contents

- 1. Project summary and product scope
- 2. MVP features and acceptance criteria
- 3. Technology stack and architecture
- 4. Java package structure
- 5. Data model and database design
- 6. AI pipeline: indexing, retrieval, and change analysis
- 7. REST API design
- 8. Security, reliability, and operational design
- 9. Four-week implementation roadmap
- 10. Evaluation and measurable quality
- 11. First three days: implementation tasks
- 12. Resume positioning and interview preparation
- 13. Final acceptance checklist
- 14. Suggested repository structure and next actions

Who this blueprint is for

A developer who already knows enterprise full-stack development and wants to demonstrate applied AI engineering, not just an LLM API wrapper. The plan deliberately favors a modular monolith and a small, measurable MVP over a broad, fragile agent platform.

Planning estimate: four weeks at 3-5 hours per day, roughly 60-100 hours if working most days. This is a target, not a guarantee; prioritize evaluation and a working end-to-end demo if the schedule slips.

1. Project summary and product scope

Recommended project: DevMind AI — an AI-powered codebase intelligence and change-risk analysis platform.

The product helps developers understand unfamiliar repositories, trace behavior across frontend/backend/database layers, assess likely impact of code changes, and identify relevant tests. It should ground answers in actual source code and be transparent when evidence is missing.

The real-world problem

- Understanding a legacy or unfamiliar codebase can require searching many files and tracing calls manually.
- Developers need to know which components and tests may be affected before changing a method or API.
- Pull-request reviews can miss indirect dependencies, compatibility risks, and missing tests.
- Generic chatbots often produce plausible explanations without verifiable repository evidence.

Flagship demo

> *A developer asks: "If I change the user authentication service, which parts of the application could be affected, what could break, and which tests should I execute?"*

- Trace relevant frontend and backend code.
- Link to real files, symbols, and line ranges.
- Show a dependency graph of potentially affected components.
- Prioritize possible risks with reasons and evidence.
- Recommend tests mapped to changed functionality.
- Explicitly list unknowns where repository evidence is insufficient.

Why this project is interview-worthy

- Demonstrates full-stack delivery: Angular, Spring Boot, database modeling, security, tests, deployment.
- Demonstrates applied AI: embeddings, hybrid retrieval, RAG, structured outputs, tool calling, and evaluation.
- Demonstrates software engineering: AST parsing, Git diffs, graph traversal, versioning, idempotency, observability.
- Provides concrete design trade-offs and benchmark results to discuss in interviews.

Scope progression

Release	Scope	Definition of done
Product 1.0	Repository intelligence and grounded Q&A;	Index a Java repository, answer questions with verifiable evidence, show file and line references.
Product 1.1	Change-impact analysis	Analyze a Git diff, find affected symbols, generate evidence-backed risk findings and test recommendations.
Product 1.2	Controlled engineering agent	Invoke narrowly scoped repository tools and prepare suggested patches for human review.

Build the first two capabilities well before attempting autonomous code changes.

2. MVP features and acceptance criteria

Build in layers. Avoid a feature checklist that expands faster than the quality of the underlying system.

Feature 1 — Repository ingestion and indexing

- Start with a trusted local repository path or a public Git repository.
- Pin each index to a commit SHA and track indexing status.
- Filter generated files, binaries, build outputs, and sensitive files.
- Parse Java classes, methods, imports, declarations, and line ranges.
- Store symbol metadata, code chunks, hashes, and source locations.
- Skip unchanged files using content hashes.

Acceptance: a sample Java repository can be indexed, and a developer can browse files and extracted symbols.

Feature 2 — Grounded codebase Q&A;

- Support questions about symbol locations, code behavior, validation, and request flows.
- Retrieve context using keyword, vector, and symbol search.
- Generate structured answers with file paths, symbols, line ranges, and uncertainties.
- Validate citations against the indexed revision before returning results.
- Show the evidence in the UI and allow users to open the relevant source.

Acceptance: a prepared question set returns inspectable evidence; unsupported questions trigger an insufficient-evidence response.

Feature 3 — Dependency and change impact

- Extract relationships such as CALLS, IMPORTS, EXTENDS, IMPLEMENTS, and REFERENCES.
- Compute a base-versus-head Git diff.
- Map changed lines to symbols and traverse verified edges within a bounded depth.
- Retrieve relevant tests and connected implementation code.
- Produce structured, prioritized risks and test recommendations.
- Separate statically verified facts from LLM-inferred risks.

Acceptance: a sample change produces a report with evidence and clearly labeled uncertainty.

Feature 4 — Controlled agent (after core MVP)

- Expose tools such as searchSymbols, findCallers, inspectDiff, and runApprovedTests.
- Validate tool arguments and enforce timeouts and iteration limits.
- Keep initial tools read-only; require approval for patches or execution.
- Record tool calls and outcomes for audit and debugging.

3. Technology stack and architecture

Layer	Recommended technology	Rationale
Frontend	Angular 19+, PrimeNG, RxJS	Use familiar strengths; build repository explorer, chat, evidence panel, dependency graph, and risk dashboard.
Backend	Java 21, Spring Boot, Spring Security	REST APIs, validation, auth, jobs, modular domain services.
AI integration	Spring AI; model adapter interface	Integrate embeddings, chat models, structured output, and tool calling while keeping provider choices replaceable.
Java parsing	JavaParser	Extract declarations and source locations.
TypeScript parsing	TypeScript Compiler API (later)	Understand Angular symbols, imports, and components after Java MVP.
Database	PostgreSQL + pgvector	Relational metadata and vector similarity search in one system.
Git	Git CLI; JGit where useful	Clone/update, commit metadata, branch comparison, diffs.
Local inference	Ollama	Run supported models locally when hardware permits.
Deployment	Docker Compose	Reproducible local stack and portfolio demo.
Testing	JUnit, Testcontainers, Angular tests	Unit, integration, API, and database verification.

Cost-conscious setup

- Use PostgreSQL + pgvector for metadata and embeddings.
- Use a local embedding model and Ollama inference if the available hardware is adequate.
- Create one configurable model adapter so a free-tier or low-cost hosted model can be substituted.
- Keep a small benchmark dataset and avoid unnecessary model calls.
- Local inference avoids per-token API billing but still consumes CPU, RAM, and possibly GPU resources. Hosted free tiers can change and may have rate limits.

Useful documentation

Spring AI: https://docs.spring.io/spring-ai/reference/
Ollama API documentation: https://github.com/ollama/ollama/blob/main/docs/api.md
Tree-sitter: https://tree-sitter.github.io/tree-sitter/

Architecture principle

Use deterministic code for deterministic tasks: parsers extract symbols, Git computes diffs, SQL queries metadata, and graph traversal follows relationships. Use the LLM to interpret evidence and explain likely risks.

High-level flow

Layer	Responsibilities
Angular UI	Repository explorer, chat, clickable evidence, dependency graph, risk report, indexing status.
Spring Boot API	Authentication, project management, DTO validation, authorization, jobs, API contracts.
Indexing pipeline	Git revision pinning, file filtering, parsing, symbol extraction, chunking, embedding persistence.
AI pipeline	Intent classification, hybrid retrieval, context expansion, reranking, structured generation, citation validation.
PostgreSQL + pgvector	Projects, revisions, files, symbols, edges, chunks, vectors, conversations, analyses, findings.
Model adapter	Local or hosted chat model and embedding model; provider-specific details isolated.

Start with synchronous calls for simple queries and a background job for indexing. Add Redis or a message broker only when you need reliable asynchronous workers or distributed processing.

4. Java package structure

Use a modular monolith with clear domain and service boundaries. Keep model-provider details behind interfaces.

```
com.devmind
■■■ domain
■   ■■■ project
■   ■■■ repository
■   ■■■ symbol
■   ■■■ analysis
■   ■■■ conversation
■■■ repository
■   ■■■ project
■   ■■■ indexing
■   ■■■ retrieval
■   ■■■ analysis
■■■ service
■   ■■■ project
■   ■■■ indexing
■   ■■■ parsing
■   ■■■ retrieval
■   ■■■ ai
■   ■■■ agent
■   ■■■ impact
■   ■■■ evaluation
■■■ dto
■   ■■■ request
■   ■■■ response
■■■ mapper
■■■ web
■   ■■■ ProjectResource
■   ■■■ RepositoryResource
■   ■■■ ChatResource
■   ■■■ AnalysisResource
■   ■■■ JobResource
■■■ config
```

Architecture rules

- Expose DTOs from web endpoints rather than persistence entities.
- Use service interfaces where they establish meaningful boundaries, especially for parsers and model providers.
- Keep parsing logic independent of Spring MVC and persistence where practical.
- Keep prompts and model-specific settings out of controllers.
- Make indexing jobs idempotent and safe to retry.
- Use database migrations and integration tests from the beginning.

Suggested interfaces

```
interface CodeParser {
    ParsedFile parse(String path, String content);
}

interface AiModelService {
    CodebaseAnswer answer(CodebaseQuestion question,
                          List<CodeEvidence> evidence);
}

interface RepositoryService {
    RepositoryRevision inspectRevision(ProjectId projectId,
                                       String commitSha);
}
```

These are illustrative interfaces. Adapt signatures to your actual domain types and error-handling strategy.

5. Data model and database design

Table	Important fields	Purpose
app_user	id, email, password_hash, created_at	User identity.
project	id, owner_id, name, repository_url, default_branch	Project and ownership.
repository_revision	id, project_id, commit_sha, indexed_at, status	Versioned index per commit.
source_file	id, revision_id, path, language, content_hash	Source file metadata.
code_symbol	id, file_id, name, qualified_name, kind, start_line, end_line	Classes, methods, functions.
code_chunk	id, file_id, symbol_id, content, start_line, end_line, embedding	Searchable source fragments.
symbol_edge	source_symbol_id, target_symbol_id, relation_type, confidence	Calls, imports, inheritance, references.
conversation	id, project_id, user_id, created_at	Chat session.
chat_message	id, conversation_id, role, content, created_at	Conversation history.
analysis_run	id, project_id, base_sha, head_sha, status, created_at	Change analysis job.
risk_finding	id, analysis_run_id, severity, category, description, evidence_json	Risk report findings.
evaluation_case	id, dataset_name, question, expected_evidence_json	Repeatable quality tests.

Data-model decisions

- **Version by commit SHA:** do not let answers accidentally cite stale source after an update.
- **Trace every chunk:** keep path, revision, symbol, and line range alongside content and embedding.
- **Typed graph edges:** distinguish verified static relationships from inferred relationships; do not treat an LLM guess as a verified call edge.
- **Constraints and indexes:** use foreign keys, unique constraints for versioned symbols, and indexes for common queries.
- **Project isolation:** scope every project query to an authorized owner.
- **Separate source from chat:** store source/indexing metadata separately and send only relevant retrieved evidence to the model.
- **Deletion:** ensure deleting a repository removes its chunks, vectors, edges, cached content, and associated data according to the retention policy.

Vector storage note

Choose an embedding model and its vector dimension before finalizing the vector column. If you change embedding models or dimensions, plan for a separate embedding version or a controlled re-embedding migration.

6. AI pipeline: indexing, retrieval, and change analysis

Pipeline A — Repository indexing

- **1. Clone/update:** pin an exact commit SHA.
- **2. Filter and parse:** exclude generated files, binaries, build outputs, and sensitive files.
- **3. Extract structure:** classes, methods, imports, calls, inheritance, and line locations.
- **4. Create chunks:** prefer symbol-aware chunks over arbitrary fixed-size splitting.
- **5. Embed and persist:** save vectors with source metadata.
- **6. Mark revision ready:** expose the index only after validation succeeds.

Use content hashes to avoid reprocessing unchanged files. A small change should not trigger full-repository embedding generation.

JavaParser is a practical starting point for Java. Add TypeScript Compiler API support after the Java workflow is reliable. Tree-sitter is an alternative for multi-language parsing.

Pipeline B — Grounded codebase Q&A;

- **1. Understand intent:** classify explain, trace, find usages, or analyze changes.
- **2. Retrieve candidates:** run keyword search, vector search, and symbol lookup.
- **3. Expand context:** follow relevant callers, imports, and related symbols with depth limits.
- **4. Rerank and limit context:** prioritize evidence while respecting token limits.
- **5. Generate structured output:** answer, evidence references, uncertainties, and suggested tests.
- **6. Validate:** verify cited paths and line ranges against the indexed revision.

Retrieval design

- **Keyword retrieval:** exact class/method names, identifiers, error messages.
- **Vector retrieval:** natural-language questions and semantic similarity.
- **Symbol/graph retrieval:** callers, imports, inheritance, and cross-layer relationships.
- Merge candidates with a simple weighted ranking or reciprocal-rank fusion; add a separate reranker only if evaluation justifies it.

Example response DTO

```
public record CodebaseAnswer(
    String answer,
    List<CodeEvidence> evidence,
    List<String> uncertainties,
    List<String> suggestedTests
) {}

public record CodeEvidence(
    String filePath,
    String symbolName,
    int startLine,
    int endLine,
    String explanation
) {}
```

This is illustrative. Validate every evidence item against the indexed revision. A valid citation is not necessarily a supporting citation, so evaluate semantic support separately.

Pipeline C — Pull request risk analysis

- Fetch base and head commits and calculate the Git diff.

- Map changed line ranges to symbols.
- Traverse verified dependency edges within a configured depth.
- Retrieve relevant implementation and test code.
- Ask the LLM for structured findings grounded in this evidence.
- Validate references, deduplicate findings, and return a report for human review.

Example risk finding

```
{
  "category": "REGRESSION_RISK",
  "severity": "MEDIUM",
  "title": "Authentication behavior may affect protected routes",
  "evidence": [{
    "filePath": "src/main/java/.../AuthService.java",
    "startLine": 42,
    "endLine": 58
  }],
  "recommendedTests": [
    "Verify valid login",
    "Verify invalid credentials",
    "Verify access to a protected route"
  ],
  "confidence": "MEDIUM"
}
```

Example only. Return a finding only when evidence supports it. Do not label an LLM suggestion a confirmed bug unless a deterministic check or reproducible test establishes that.

7. REST API design

Method	Endpoint	Responsibility
POST	/api/projects	Create project.
GET	/api/projects	List authorized projects.
POST	/api/projects/{id}/index-jobs	Start indexing.
GET	/api/jobs/{jobId}	Check indexing status.
GET	/api/projects/{id}/files	Browse indexed files.
GET	/api/projects/{id}/symbols/search?q=	Search symbols.
POST	/api/projects/{id}/chat	Ask a grounded code question.
POST	/api/projects/{id}/analyses	Start change-impact analysis.
GET	/api/analyses/{analysisId}	Fetch analysis status/results.
GET	/api/analyses/{analysisId}/findings	Retrieve risk findings.
GET	/api/projects/{id}/evaluations/latest	View latest benchmark results.

Example chat request

```
{
  "question": "Trace the authentication flow",
  "conversationId": null,
  "revision": "a1b2c3d4"
}
```

Example chat response

```
{
  "answer": "The authentication flow begins in ...",
  "revision": "a1b2c3d4",
  "evidence": [{
    "filePath": "src/main/java/.../AuthResource.java",
    "startLine": 35,
    "endLine": 48,
    "symbolName": "authenticate"
  }],
  "uncertainties": [],
  "suggestedTests": [
    "Test successful authentication",
    "Test invalid credentials"
  ]
}
```

Paths and line numbers above are examples, not output from a real repository.

API design guidance

- Use 202 Accepted plus a job ID for long-running indexing and analysis tasks.
- Persist job status and allow Angular to poll a status endpoint.
- Use consistent error responses, pagination, input validation, timeouts, and correlation IDs.
- Add idempotency and retry handling to background jobs.
- Use revision-aware requests so the UI knows which commit an answer describes.

8. Security, reliability, and operational design

- **Repository access:** begin with public repositories or local repositories explicitly owned by the developer. Do not collect personal Git credentials.
- **Authorization:** enforce project ownership for every API, file, conversation, and analysis query.
- **Prompt injection:** treat comments, README files, and source code as untrusted data, not instructions that override system policy.
- **Secret handling:** exclude .env files, private keys, tokens, and other sensitive files by default; scan before model submission.
- **Tool restrictions:** allow read-only operations first. Never convert model-generated strings into unrestricted shell commands.
- **Sandboxing:** if tests or patches are added later, isolate execution, restrict network access, enforce timeouts, and limit CPU, memory, and disk.
- **Human approval:** require explicit approval before modifying files, creating commits, or performing destructive operations.
- **Data retention:** repository deletion should remove associated chunks, embeddings, graph edges, conversations, and caches according to policy.
- **Observability:** log model name, latency, token usage where available, retrieval IDs, tool calls, and validation outcomes without unnecessarily logging source or secrets.

Reliability practices

- Pin model and embedding versions for benchmark runs.
- Make indexing idempotent and safe to retry.
- Limit retrieval size, graph traversal depth, tool calls, and agent iterations.
- Use timeouts and graceful failure when a model is unavailable.
- Return partial job status and actionable error messages.
- Keep test data and public sample repositories separate from confidential client code.

Initial product should answer questions and analyze diffs, not automatically modify a user's repository.

9. Four-week implementation roadmap

Estimated total: 60-100 hours at 3-5 hours per day, assuming work on most days. Adjust the scope if parsing or evaluation takes longer than expected.

Week 1 — Foundation (15-25 hours)

- Create Spring Boot and Angular projects; run PostgreSQL + pgvector in Docker Compose.
- Create project, revision, source-file, symbol, and indexing-job entities and migrations.
- Implement safe local repository ingestion and commit identification.
- Parse Java classes, methods, imports, and line ranges.
- Build a basic UI for indexing status, files, and symbols.

Exit: index a sample Java repository and browse extracted symbols.

Week 2 — AI retrieval (15-25 hours)

- Integrate Ollama or a hosted model through a provider abstraction.
- Generate embeddings and persist them in pgvector.
- Implement keyword + vector retrieval and merge candidates.
- Build chat API with structured answers and source references.
- Create Angular chat view with clickable citations and insufficient-evidence state.

Exit: answer prepared repository questions and inspect cited code.

Week 3 — Change impact (15-25 hours)

- Add symbol relationships and bounded dependency traversal.
- Implement base-versus-head Git diff analysis.
- Map changed lines to symbols and retrieve relevant dependencies.
- Generate structured risk findings and test recommendations.
- Display affected-code graph, risk categories, and evidence in Angular.

Exit: analyze a real code change and distinguish facts from possible risks.

Week 4 — Quality and demo (15-25 hours)

- Build a repeatable evaluation dataset and calculate baseline metrics.
- Add integration tests, authorization checks, timeouts, and error handling.
- Add bounded tool calling for symbol search and dependency inspection.
- Deploy the stack and document architecture and trade-offs.
- Record a demo and publish a README with measured results and limitations.

Exit: another developer can run the project, reproduce the benchmark, and understand limitations.

If the schedule slips, drop advanced graph visualization or autonomous patching before sacrificing evaluation, security, or the end-to-end demo.

10. Evaluation and measurable quality

Build a benchmark before tuning the system. Create around 50 questions over one or two public repositories you understand. For each question, record expected relevant files/symbols and supporting evidence. Review ambiguous answers before including them.

Question category	Example	What it tests
Symbol lookup	Where is the login endpoint implemented?	Exact retrieval.
Code explanation	How is a request validated?	Contextual understanding.
Cross-layer tracing	How does the UI request reach the database?	Multi-hop retrieval.
Dependency analysis	Which methods call this service?	Static analysis.
Change impact	What could be affected by this diff?	Dependency reasoning.
Insufficient evidence	Does this code send an email?	Appropriate abstention.

Metrics

Metric	Definition / use
Retrieval Recall@5	Questions where at least one expected relevant item appears in the top five retrieved results, divided by total questions.
Citation precision	Proportion of checked citations that genuinely support their associated claims; correct file but irrelevant lines do not count.
Answer correctness	Developer-reviewed scores for factual correctness, completeness, and unsupported claims.
Risk finding precision	Proportion of reviewed findings judged actionable and genuinely relevant; track false positives separately.
Latency and cost	Median and p95 response time, model calls per request, token usage where available, and estimated cost per analysis.

Three controlled experiments

- **A. Vector-only versus hybrid retrieval:** same questions and model; compare retrieval quality and answer correctness.
- **B. Retrieved chunks versus graph-expanded context:** evaluate cross-layer questions and check whether expansion improves completeness without irrelevant context.
- **C. Unvalidated versus validated citations:** measure invalid or stale citations caught before UI delivery.

Keep model settings consistent, use the same benchmark dataset, record model and index revisions, and report actual results including failures. Save inputs, outputs, retrieval settings, and evaluation code for repeatability.

11. First three days: implementation tasks

Do not start by building the chat UI. Start by making the code-indexing pipeline work end to end.

Day 1 — Foundation

- Create a Git repository named devmind-ai.
- Generate Spring Boot backend with Java 21 and create the Angular frontend.
- Add PostgreSQL and pgvector to Docker Compose.
- Create the first database migration.
- Add a health endpoint and verify the full stack starts.

Deliverable: a running, reproducible application skeleton.

Day 2 — Indexing

- Accept a local repository path from trusted configuration.
- Validate that the path is inside an explicitly permitted root.
- Read tracked source files while excluding generated output and sensitive files.
- Parse Java classes and methods.
- Store file paths, symbol names, and line ranges in PostgreSQL.
- Add integration tests for parsing and persistence.

Deliverable: a command or API that indexes a Java repository and persists its symbols.

Day 3 — Retrieval

- Implement a symbol-search endpoint.
- Add code chunks with stable source references.
- Integrate an embedding model.
- Persist vectors and implement the first similarity search.
- Test exact-name and natural-language queries.
- Record retrieved evidence for each test query.

Deliverable: a working retrieval pipeline, ready for grounded AI answers.

First milestone

Given a repository and a class name, return the correct file, qualified symbol name, line range, and relevant source text. Once that works, add embeddings and an LLM.

12. Resume positioning and interview preparation

Use the following as a target resume description after implementation. Remove features you have not completed and add measured results only after you have evaluated the system.

DevMind AI — Codebase Intelligence & Change Risk Analysis Platform

Technologies: Angular, Java 21, Spring Boot, Spring AI, PostgreSQL, pgvector, Docker

- Designed and developed an AI-powered developer platform that analyzes code repositories, retrieves relevant source code, and answers architectural and implementation questions with verifiable code references.
- Implemented hybrid retrieval combining semantic search, keyword matching, and code dependency analysis to improve contextual understanding across full-stack applications.
- Built an AI-driven pull request risk analyzer that traces code dependencies, identifies potential regression risks, and recommends targeted test cases.
- Developed a tool-calling workflow for repository exploration and code analysis, with controlled execution, validation, and human approval for proposed changes.
- Established an evaluation pipeline to measure retrieval quality, answer correctness, citation accuracy, response latency, and inference cost.

Interview questions the project should prepare you for

Question	Evidence from your implementation
Why RAG instead of fine-tuning?	Fresh repository data, source grounding, lower-cost updates.
How do you reduce hallucinations?	Evidence retrieval, citation validation, abstention, and evaluation.
Why hybrid search?	Exact identifiers, semantic queries, and retrieval benchmarks.
What makes an AI agent different from a chatbot?	Tool selection, execution, observation, bounded iteration, stopping conditions.
How do you handle large repositories?	Incremental indexing, content hashes, pagination, bounded retrieval, caching.
How do you control LLM costs?	Context limits, model routing, caching, incremental embeddings, call budgets.
How do you secure an AI agent?	Least privilege, validation, sandboxing, approval gates, auditability.
How would you scale the system?	Background jobs, queues, idempotency, worker concurrency, DB indexes.
How do you know the AI works?	Benchmarks, baselines, retrieval metrics, citation quality, regression tests.
Why a modular monolith?	Lower operational complexity now, clear boundaries for future extraction.

In interviews, explain what you chose, alternatives you considered, why you rejected them, and what evidence would make you change the design.

13. Final acceptance checklist

- A developer can run the stack using documented instructions.
- A repository can be indexed against a specific commit SHA.
- Java symbols and source locations are searchable.
- AI answers include verifiable citations or acknowledge missing evidence.
- The benchmark can be rerun and results reproduced.
- A Git diff produces an evidence-backed change-impact report.
- Project-level authorization and repository input validation are tested.
- The demo shows one complete workflow without manual database edits.
- The README documents architecture, trade-offs, limitations, and actual metrics.

Readiness standard

The project is portfolio-ready when another developer can run it, reproduce the benchmark, inspect evidence for an answer, and understand where the system can fail. A polished screenshot is useful, but reproducibility and honest evaluation are stronger proof.

14. Suggested repository structure and next actions

```
devmind-ai/
■■■ backend/
■   ■■■ src/main/java/com/devmind/
■   ■■■ src/main/resources/
■   ■   ■■■ db/migration/
■   ■   ■■■ application.yml
■   ■■■ src/test/
■■■ frontend/
■   ■■■ src/app/
■   ■   ■■■ projects/
■   ■   ■■■ repository-explorer/
■   ■   ■■■ code-chat/
■   ■   ■■■ impact-analysis/
■   ■   ■■■ shared/
■   ■■■ package.json
■■■ evaluation/
■   ■■■ datasets/
■   ■■■ scripts/
■   ■■■ reports/
■■■ docs/
■   ■■■ architecture.md
■   ■■■ threat-model.md
■   ■■■ evaluation.md
■   ■■■ demo-script.md
■■■ docker-compose.yml
■■■ README.md
■■■ .gitignore
```

Recommended first implementation backlog

Priority	Task	Done when
P0	Create repository and Docker Compose stack	Backend, frontend, and database start reproducibly.
P0	Create schema and migrations	Project, revision, file, symbol, and job records persist.
P0	Implement safe repository ingestion	Only permitted source files are processed.
P0	Implement Java parsing	Symbols and source line ranges are stored and searchable.
P1	Add embeddings and pgvector	Semantic retrieval returns source-backed candidates.
P1	Add grounded Q&A;	Responses contain validated evidence and uncertainties.
P1	Create benchmark dataset	Quality can be measured repeatedly.
P2	Add Git diff and impact analysis	Changed symbols and likely dependencies are reported.
P2	Add controlled tools and deployment polish	Demo is reproducible and actions remain bounded.

Final recommendation

Build Java repository indexing and grounded Q&A; first. Then add change-impact analysis. Code parsing, reliable RAG, and evidence-backed risk analysis are enough to create a serious portfolio project; agentic execution should be the final differentiator, not the starting point.

Quality bar: be honest about what is implemented, show actual measurements, explain failure modes, and keep the entire demo reproducible.

End of blueprint | DevMind AI | Implementation plan prepared for portfolio development